

Student Management System Using SQL

Project Report

Submitted By

Name: Varsha Sanmuhi G

Course: Data Analysis with SQL

Date of Submission: 5.8.26

Table of Contents

1. Introduction

2. Project Requirements & Scope

3. Database Architecture & 3NF Schema Design

4. Advanced Programmable Automation Layers

5. Performance Optimization & Index Engineering

6. Business Intelligence Mapping & KPI Design

7. Conclusion

Introduction

This project delivers a robust, end-to-end, enterprise-level relational database and analytics engine engineered to handle high-throughput corporate transactions. Built using Third Normal Form (3NF) relational standards, the architecture optimizes data storage integrity while eliminating operational update anomalies.

The system features advanced programmable logic layers—including automated inventory auditing triggers and scalable pricing stored procedures—complemented by structural B-Tree indexing arrays that minimize disk read latency. The data extraction layer computes business metrics directly on the server database layer using optimized database views. These pre-computed views expose clean data channels optimized for seamless integration with front-end Business Intelligence visualization suites like Power BI or Tableau.

Project Requirements & Scope

The system addresses eight core enterprise data engineering tasks to establish an automated operational pipeline:

1. **Normalized Database Design:** Build a relational schema that minimizes data redundancy.
2. **High-Volume Scaling:** Load baseline test data sets simulating actual retail business loops.
3. **Advanced SQL Automation:** Deploy automated triggers and administrative procedures.
4. **Performance Optimization:** Tuned table access paths using execution benchmarking indexes.
5. **Business Intelligence KPIs:** Calculate gross margins, sales velocity, and regional trends.
6. **Analytical Controls:** Leverage advanced window ranking metrics to track consumer cohorts.
7. **BI Tool Integration:** Document connectivity mappings for analytical dashboards.
8. **Architecture Governance:** Formalize deployment sequences and data workflows.

Database Architecture & 3NF Schema Design

To guarantee absolute data consistency across high-frequency transactional pipelines, the system uses a modular star-adjacent schema optimized to full Third Normal Form (3NF) standards.

Entity Relational Breakdown:

- **customers:** Holds isolated customer registration metrics and geographic demographics.
- **products:** Represents the master catalog, tracking wholesale cost prices, retail prices, and live stock velocity.
- **orders:** Serves as the high-level transactional invoice header, isolating date markers and logistical fulfillment tracking.
- **order_items:** A highly granular bridge matrix table resolving the many-to-many relationship mapping rules linking specific invoices to multiple catalog products without cell duplication.
- **inventory_logs:** An isolated audit ledger table populated asynchronously via database automation layer scripts.

Advanced Programmable Automation Layers

Rather than relying on resource-intensive application-level computing, critical enterprise operational workflows are executed directly within the database management runtime layer.

A. Automated Supply Chain Audit Trigger (trg_track_inventory)

Monitors inventory level modifications dynamically. The exact millisecond an order transaction modifies any item's stock quantity, the trigger calculates the inventory shift, captures the absolute historic stock versus the new stock, and writes a tamper-proof audit record directly into the inventory_logs table.

B. Bulk Price Calibration Stored Procedure (prc_adjust_category_prices)

Provides administrators with an automated utility to execute global pricing revisions across entire catalog groupings simultaneously (e.g., executing a standard 10% inflation adjustment across all "Software" products) using parameterized safe-decimal percentage multipliers.

Performance Optimization & Index Engineering

To prevent sluggish system bottlenecks during complex multi-table analytical joins across large data volumes, dedicated custom **B-Tree Indexes** are injected into primary query filters:

- **idx_orders_customer_id**: Accelerates transactional profile joins linking invoices to customer accounts.
- **idx_orders_order_date**: Optimizes time-series partitioning, filtering, and chronological sorting loops.
- **idx_order_items_product_id**: Streamlines individual unit sales velocity tracing.
- **idx_products_category**: Speeds up high-level executive categorical reporting aggregations.

Execution Path Verification (Explain Benchmarking Result)

Running diagnostic checks via the EXPLAIN query engine confirms that index pathways successfully replace slow, resource-heavy, full-disk sweeps with highly efficient nested-loop algorithm steps, yielding a minimized processing execution cost structure (cost=7.1, rows=6).

Business Intelligence Mapping & KPI Design

The reporting layer is fully decoupled from front-end visual canvas files, processing intensive data computations strictly on the database side through permanent view layers:

A. View: vw_bi_sales_performance_kpis

- **Visual Chart Target**: Clustered Column Bar Graph
- **Computed Business Insights**: Tracks absolute total orders processed, unit sales velocity, gross revenue (SUM(quantity * unit_price)), and true net operational profit margin configurations (SUM(quantity * (sale_price - cost_price))).

B. View: vw_bi_customer_clv_analytics

- **Visual Chart Target**: Geographic Regional Heatmaps / Cohort Standings
- **Computed Business Insights**: Employs advanced SQL window partitioning metrics (RANK() OVER (PARTITION BY region ORDER BY lifetime_value DESC)) to segment and score target customer lifetime values dynamically inside their respective global sales sectors.

Conclusion

The **Enterprise Business Intelligence Database System** successfully achieves all target operational benchmarks. By handling complex data normalizations, background tracking workflows, and heavy data metric aggregations entirely within the optimized SQL relational layer, the platform minimizes front-end data strain and guarantees that all enterprise metrics remain standard across every company department.

Future Scaling Recommendations:

- 1. Partition Tuning:** For multi-million row matrices, introduce explicit chronological table range partitioning on the `order_date` parameter to further prune index scan trees.
- 2. Semi-Structured Storage:** Incorporate secondary unstructured JSON data cells into the products table schema to store variable product specifications without breaking standard relational boundaries.